

CS354 - ADVANCED ALGORITHMS

PROJECT REPORT

The Use of a One-Stage Dynamic Programming Algorithm for Connected Word Recognition

Made By -

Sai Kiran Sajja (21CSB0A49)

Ganesh Yadava (21CSB0A66)

Dhruv Aggarwal (21CSB0F05)

Under the Guidance of -

Dr. Manish Kumar Bajpai

Introduction

Problem Definition :

Connected word recognition aims to accurately identify and transcribe spoken words from an unknown input pattern consisting of a sequence of time frames. The challenge lies in efficiently aligning the input pattern with reference templates of individual words, chosen from a predefined vocabulary, to determine the sequence of words that best matches the input.

This project demonstrates one-stage dynamic programming using synthetic feature vectors. It jointly aligns templates and detects word boundaries. Audio loading, speech-feature extraction, coarticulation models and silence handling are outside the current implementation. The algorithm aims to:

- Align the input pattern with reference word templates to find the best matching sequence of words.
- Handle variations in speaking rate and pronunciation by allowing for time warping between the input pattern and templates.
- Automatically detect word boundaries and recognize complete words without the need for explicit segmentation rules.
- Support future extensions with syntactic constraints; these are not implemented in this demonstration.

The solution should leverage dynamic programming techniques to optimize the alignment process, ensuring that the recognized word sequence accurately reflects the input utterance.

The implemented program recovers a minimum-distance sequence of supplied word templates. Its synthetic examples verify alignment behavior; they do not establish accuracy on real speech or robustness to environmental noise.

Broader speech recognition applications include command recognition, dictation and captioning. These motivate the problem; the examples below are not claims that commercial systems use this particular implementation.

- CWR allows voice assistants like Siri or Alexa to understand spoken commands like "play music" or "search for weather."
- CWR enables dictation software to convert spoken words into written text, useful for journalists, students, or creating captions.
- Virtual assistants like Google Assistant leverage CWR to understand and respond to natural language queries.
- CWR can be used in automatic captioning systems to generate captions for videos or live broadcasts from spoken content.

Algorithm

One-Stage Dynamic Programming Algorithm for Connected Word Recognition

This algorithm addresses the problem of connected word recognition (CWR) using dynamic programming. It offers an efficient approach to recognize a sequence of spoken words uttered together without silent pauses. Here's a breakdown of the algorithm description and key terms for a research paper:

Formulation :

- The algorithm operates on pre-extracted features representing the input speech signal and reference patterns for each word in the vocabulary. Common features include Mel-frequency cepstral coefficients (MFCCs).
- Let T be the length of the input feature sequence and K be the number of words in the vocabulary.
- Each reference word has a corresponding reference pattern, denoted as R_k ($k = 1, 2, \dots, K$), where $R_k(j)$ represents the feature vector for the j -th frame of the k -th reference pattern.

- $D(i, j, k)$ denotes the minimum accumulated distance for any path reaching the grid point (i, j, k) in the dynamic programming search space.
- $d(i, j, k)$ represents the local distance between the i -th frame of the input feature sequence and the j -th frame of the k -th reference pattern. The implementation uses Euclidean distance between feature vectors. Dynamic Time Warping (DTW) is an alignment method, not a local feature distance.

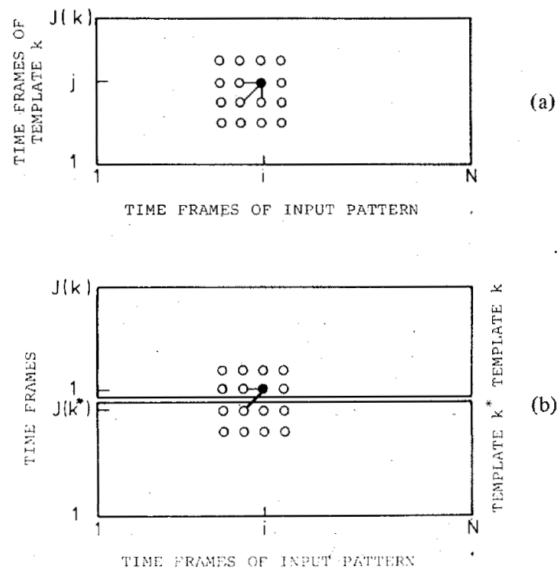


Fig. 2. (a) Within-template transition rules. (b) Illustration of between-template transition rules.

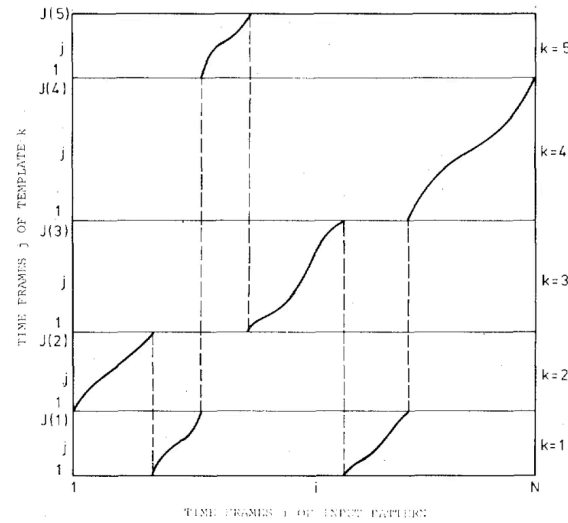


Fig. 1. The connected word recognition problem. The optimal path provides the unknown sequence of words as well as the nonlinear time alignment between the corresponding sequence of templates and the input pattern.

Algorithm Description:

1. **Initialization:** Mark unreachable states as infinity. Set $D(1, 1, k)$ to its local distance for each template. For later template frames at $i=1$, accumulate horizontally from the previous template frame. Only add local distances to finite predecessors.
2. **Dynamic Programming Iteration:** Iterate through remaining time frames ($i=2$ to T) and reference patterns ($k=1$ to K):
 - For the first frame of each reference pattern ($j=1$), use the "between-template" transition rule to consider the ending frame of any previous reference pattern as a possible predecessor.

- For remaining frames within a template ($j=2$ to $J(k)$, where $J(k)$ is the length of the k -th reference pattern), use the "within-template" transition rule to consider the previous frame of the same reference pattern as a possible predecessor.
3. **Recursion:** The core of the algorithm lies in the dynamic programming recursion used to calculate $D(i, j, k)$. This recursion considers two types of transitions:
- **Within-Template Transition (Eq. 4a):** Take the minimum over three predecessors in the same template: $(i-1, j, k)$, $(i-1, j-1, k)$, and $(i, j-1, k)$. Add the local distance. These moves accommodate timing differences while preserving frame order.
 - **Between-Template Transition (Eq. 4b):** At $j=1$, compare staying at the first template frame, $D(i-1, 1, k)$, with the cheapest completed template $D(i-1, J(k^*), k^*)$ over all k^* . Add the local distance and record whether the transition begins a new word.
4. **Backtracking:** Select the cheapest completed template at the last input frame. Follow saved predecessors, record words at saved start boundaries, and reverse the collected words. This preserves consecutive repetitions of the same word.

$$D(i, j, k) = d(i, j, k) + \min \{D(i-1, j, k), D(i-1, j-1, k), D(i, j-1, k)\}. \quad (4a)$$

$$D(i, 1, k) = d(i, 1, k) + \min \{D(i-1, 1, k); D(i-1, J(k^*), k^*): k^* = 1, \dots, K\}. \quad (4b)$$

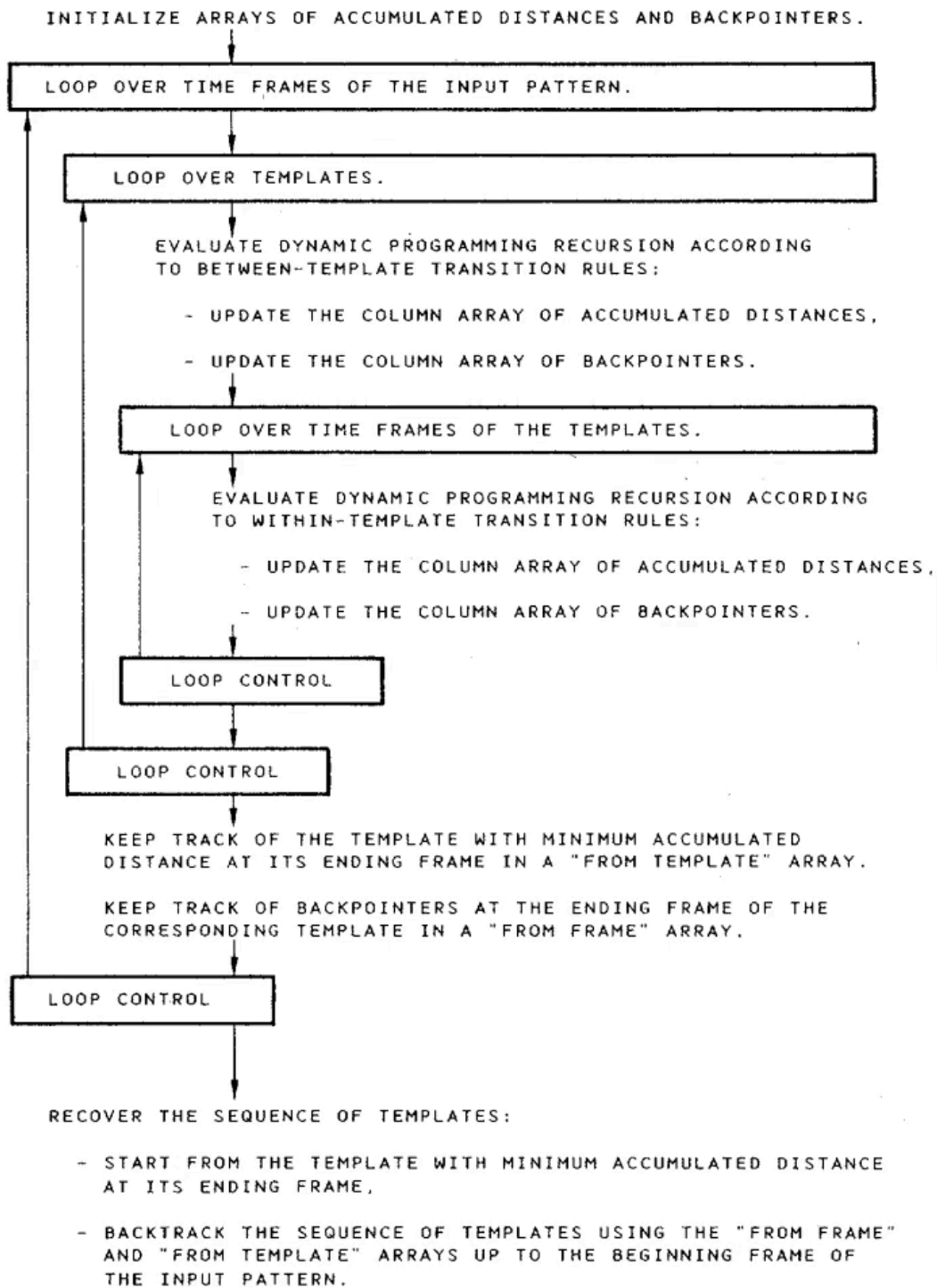


Fig.6. Schematic diagram of the one-stage dynamic programming algorithm.

Algorithm Analysis :

The corrected program takes $O(N * S * F)$ time, where S is the sum of all template lengths and F is the feature dimension. For fixed F , this is $O(N * S)$, bounded by $O(N * K * J)$, where:

- N is the length of the test pattern,
- K is the number of templates, and
- J is the maximum length of a template.

Each input frame visits every template frame once. The best completed template is cached once per input frame, avoiding repeated vocabulary scans at word starts.

The distance table and saved predecessors require $O(N * S)$ storage. These predecessors support complete traceback.

Implementation

We implemented the recurrence in standard C++17 using variable-length templates and floating-point Euclidean distances. The current source is `AA_Project_Code.cpp` in this project folder. The archived sharing link below may contain the earlier version.

Link for the code : https://drive.google.com/drive/folders/1jj5_cYrBbXpXmyCUrZ31SG-c1tiMmKi7?usp=sharing

The demonstration uses 6 input frames and 2 templates with 2 features per frame.

ONE: (0,0), (1,0), (2,0)

TWO: (8,0), (9,0)

Input: (0,0), (1,0), (1,0), (2,0), (8,0), (9,0)

Template lengths are 3 and 2. The repeated input frame demonstrates time stretching.

Verified output from the corrected program:

Recognized words: ONE TWO

Total distance: 0

All demonstration checks passed.

Validation

Time-stretched ONE TWO: expected words and zero cost passed.
Repeated ONE ONE: word-boundary check passed.
Compressed alignment: expected cost 1 passed.
Fractional feature distance: expected cost 0.5 passed.
Mismatched feature dimensions: rejected as expected.
C++17 compilation and address/undefined-behavior sanitizer checks passed on these examples.

Comparison with other approaches

TABLE 1
COMPUTATIONAL COMPARISONS OF DYNAMIC PROGRAMMING ALGORITHMS
FOR CONNECTED WORD RECOGNITION AND TYPICAL COMPUTATIONAL
REQUIREMENTS FOR VOICE DIALING (i.e., 12 DIGITS IN A STRING)

	Two-Level Algorithm	Level Building Algorithm	Reduced Level Building Algorithm	One-Stage Algorithm
Number of basic time warps	$K \cdot N$	$K \cdot M$	$K \cdot M$	K
Size of time warps	$\bar{J} \cdot (2R + 1)$	$\bar{J} \cdot N/3$	$\bar{J} \cdot (2R + 1)$	$\bar{J} \cdot N$
Total computation	$K \cdot N \cdot \bar{J} \cdot (2R + 1)$	$K \cdot M \cdot \bar{J} \cdot N/3$	$K \cdot M \cdot \bar{J} \cdot (2R + 1)$	$K \cdot \bar{J} \cdot N$
Storage	$2 \cdot N \cdot (2R + 1)$	$3 \cdot N \cdot M$	$3 \cdot N \cdot M$	$2 \cdot (N + K \cdot \bar{J})$
Number of basic time warps	3600	120	120	10
Size of time warps	875	4200	875	12600
Total computation	3150000	504000	105000	126000
Storage	18000	12960	12960	1420

where $\bar{J} = 35$ = average length of a template
 $K = 10$ = number of templates
 $M = 12$ = maximum number of words in the input string, e.g., for voice dialing (5 + 7) digits
 $N = 360$ = length of the input string
 $R = 12$ = range parameter for time warping

Source: The Use of a One-Stage Dynamic Programming Algorithm for Connected Word Recognition, Table I. The reproduced table reports the paper's comparison under its stated assumptions, not measurements from our demonstration. It compares four algorithms:

- **Two-Level Algorithm (Sakoe):** This approach performs time warping for each input frame and reference pattern.

- **Level Building Algorithm (Myers & Rabiner):** This method iteratively builds levels based on a pre-specified maximum number of words.
- **Reduced Level Building Algorithm (Myers & Rabiner):** A variation of the level building algorithm with a restricted search region.
- **One-Stage Dynamic Programming Algorithm:** The algorithm presented in the paper, which performs dynamic programming with a single global time warp.

The comparison focuses on scenarios without syntactic constraints on word sequences.

Metrics:

- **Computational Requirements:** Measured as the number of local distance calculations, which reflects the overall processing cost.
- **Storage Requirements:** Refers to the memory needed for storing intermediate results during recognition.

Limitations

- This demonstration uses synthetic features. It does not load audio, extract MFCCs, model silence or coarticulation, or enforce syntax constraints. No real-speech accuracy or runtime benchmark has been measured.
- The dynamic programming approach utilizes multiple arrays to store accumulated distances and backpointers. This can lead to increased memory usage, especially for large vocabularies.

Future Improvements

To improve the one-stage dynamic programming algorithm for connected word recognition, several improvisations can be explored. Speed optimizations like early abandonment and beam search can reduce computation. Additionally, incorporating language models and phonetic constraints can utilize contextual

information and restrict the search space, potentially leading to more robust recognition in real-world scenarios.